



PHP Practicum



LARAVEL FRAMEWORK

WGS LAKSHITHA

Laravel Framework

- Starting by installing Laravel

```
% composer create-project laravel/laravel my-sample
```

- A directory named `sample` will be created, so navigate to this directory

```
% cd my-sample
```

Laravel Framework

- Laravel Project Directory Structure

```
— sample
  |— app # Application logic
  |— bootstrap # Laravel framework startup code
  |— config # Configuration files
  |— database # DB-related files like Migration files
  |— public # Document root for the web server
  |— resources # View files and language translation files
  |— routes # Routing files
  |— storage # Files used by the framework
  |— tests # Test code
  └— vendor # Libraries installed via Composer
```

Laravel Framework

The structure may vary depending on the Laravel version.

You will build your web application according to this structure. Before we start, let's briefly introduce some important files.

.env File - This file is used to write the environment configuration information for your application.

Artisan File - The artisan file is the executable for Laravel's command-line tool, Artisan. In Laravel, you use Artisan commands to perform various operations.

Laravel Framework

- **Serve Command** - This command runs your Laravel project using PHP's built-in server.

```
% php artisan serve - host=localhost - port=8000
```

- **Route List Command** - This command displays a list of all routing definitions.

```
% php artisan route:list
```

Database Configuration

After creating a Laravel project, the first thing you need to do is set up the database. Initially, you will need to perform the following tasks:

- Connection settings
- Migration
- Model creation
- Seeding

Connection Settings

The database connection information is written in the ``.env`` file. The default values are as follows. Configure them according to your environment.

DB_CONNECTION=mysql

DB_HOST=127.0.0.1

DB_PORT=3306

DB_DATABASE=test

DB_USERNAME=admin

DB_PASSWORD=pass

Migration

Migration is a system that manages the creation and modification of tables using PHP source code instead of SQL. Follow these steps to perform migrations:

1. Create a migration file
2. Run the migration

Creating a Migration File

- To create a migration file, use Artisan's `make:migration` command. You can name the file anything you like, but it's good practice to choose a name that indicates what operation the migration performs. Let's create a migration file for a *Books* table.

```
php artisan make:migration create_books_table
```

- This command generates a migration file in the `database/migrations` directory with a timestamp in its filename. You can then edit this file to define the structure of the *Books* table. The migration file will be created in the `database/migrations` directory.

```
% php artisan make:migration create_books_table - create=books
```

Migration file

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

class CreateBooksTable extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('books', function (Blueprint $table) {
            $table->id(); // AUTO_INCREMENT PRIMARY KEY
            $table->string('name', 50)->nullable(false); // VARCHAR(50) NOT NULL
            $table->integer('price')->nullable(false); // INT NOT NULL
            $table->string('author', 50)->nullable(); // VARCHAR(50)
            $table->timestamps(); // created_at and updated_at timestamps
        });
    }

    /**
     * Reverse the migrations.
     *
     * @return void
     */
    public function down()
    {
        Schema::dropIfExists('books');
    }
}
```

Running Migrations

- To run migrations, use the Artisan migrate command.

```
% php artisan migrate
```

Creating Models

- Models are objects mapped to tables in the database. They serve as classes for performing database operations.

```
% php artisan make:model Book
```

```
<?php

namespace App;

use Illuminate\Database\Eloquent\Model;

class Book extends Model
{
    //
}
```

Seeding

- Seeding is the process of inserting test data or master data records that are needed when the application starts. It is done using commands.

```
% php artisan make:model Book
```

- You can create a seeder file using Artisan's `make:seeder` command:

```
% php artisan make:seeder BooksTableSeeder
```

Seeding

```
<?php

use Illuminate\Database\Seeder;

class BooksTableSeeder extends Seeder
{
    /**
     * Run the database seeds.
     *
     * @return void
     */
    public function run()
    {
        //
    }
}
```

Routing

- Once the database setup is complete, we can start building the application. Let's begin with routing.

```
% php artisan make:model Book
```

- You can create a seeder file using Artisan's `make:seeder` command:

```
% php artisan make:seeder BooksTableSeeder
```

```
#### `routes/web.php`  
  
<?php  
Route::get('/', function () {  
    return view('welcome');  
});
```

Creating a Controller

```
#### `app/Http/Controllers/BookController.php`

<?php
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Http\Controllers\Controller;
use App\Book;

class BookController extends Controller
{
    public function index()
    {
        // Retrieve all values from the Book table
        $books = Book::all();

        // Pass the retrieved values to the "book/index" view
        return view('book/index', compact('books'));
    }

    public function edit($id)
    {
        // Retrieve the Book information with the same ID as the URI parameter
        $book = Book::findOrFail($id);

        // Pass the retrieved values to the "book/edit" view
        return view('book/edit', compact('book'));
    }
}
```

% php artisan make:controller BookController

Thank you!

